PARALLEL BRANCH-AND-BOUND FOR EXACT PUMP SCHEDULING WITH CONSTANT-TIME HYDRAULIC BACKTRACKING

Jéssica Gomes Melo da Rocha[1], Albert Einstein Fernandes Muritiba[1], Ascânio Dias Araújo[1,2], Carlile Lavor[3], and Michael Souza[1,3]

[1]Dept. of Statistics and Applied Mathematics, Federal University of Ceará, Campus do Pici, Fortaleza, CE 60455-760, Brazil. Corresponding author. Email: michael@ufc.br

[2]Dept. of Physics, Federal University of Ceará, Campus do Pici, Fortaleza, CE 60455-760, Brazil.

[3]IMECC, University of Campinas, Rua Sérgio Buarque de Holanda, 651, Campinas, SP 13083-859, Brazil.

ABSTRACT

Exact pump scheduling is computationally difficult because the number of binary schedules grows exponentially and each candidate requires hydraulic simulation. We present a distributed-memory parallel Branch-and-Bound framework for hourly scheduling of interchangeable fixed-speed pumps. The method searches over aggregate pump counts, enforces individual actuation limits through dynamic programming, and uses hydraulic snapshots to avoid repeated prefix simulation. Experiments on a benchmark network show that the framework completes the exact search under different actuation limits and benefits substantially from cost pruning, state persistence, and parallel execution. Relaxing actuation limits reduces operating cost but increases the computational effort required for certification.

PRACTICAL APPLICATIONS

Water utilities must decide when to run pumps so that storage tanks remain available, customers receive adequate pressure, and energy costs stay under control. The proposed method can help operators and consultants compare daily pumping strategies under these competing requirements

and identify the lowest-cost schedule under the modeled conditions. It can also show the cost of limiting pump starts and stops, check whether a proposed schedule respects pressure and tank-level requirements, and provide a reliable reference for evaluating faster scheduling tools. These capabilities are useful for day-ahead planning, operational policy studies, and investment analyses that consider whether additional storage or pumping flexibility would be valuable. The proposed formulation is intended for pump groups that can be treated as interchangeable and does not directly cover stations with different pump characteristics. The results presented here come from one small benchmark network, so they do not establish how quickly the method will perform on larger or real-world networks. The method is therefore best viewed as a decision-support and benchmarking tool.

INTRODUCTION

Pumping operations constitute the primary energy expenditure in water distribution networks (WDNs). Storage tanks provide temporal flexibility by decoupling water supply from real-time consumer demand, enabling operators to shift pumping schedules toward hours with lower electricity tariffs while maintaining required junction pressures and service availability (Reis et al. 2023; Paola et al. 2025). Operational schedules must also constrain the frequency of pump starts and stops to prevent excessive mechanical wear and reduce equipment degradation (Lansey and Awumah 1994; Costa et al. 2016).

Exploiting this operational flexibility requires solving a challenging combinatorial optimization problem. For a system with N_p fixed-speed pumps and T discrete scheduling intervals, the decision space contains $2^{N_p \times T}$ possible binary operation schedules. Each schedule induces a complex non-linear hydraulic trajectory governed by pipe friction head losses, pump performance curves, time-varying consumer demands, and dynamic tank storage. Furthermore, pressure limits, tank operating bounds, daily volume recovery, and actuation constraints couple decisions across the planning horizon. When formulated with explicit hydraulic constraints, the problem constitutes a non-convex mixed-integer non-linear program (MINLP) that is NP-hard in general (Fooladivanda and Taylor 2018; Verleye and Aghezzaf 2016; Bonvin et al. 2021).

To address this computational complexity, much of the literature combines hydraulic simulation with metaheuristic search algorithms. Genetic algorithms, ant colony optimization, particle swarm methods, and harmony search variants accommodate discrete binary decisions and non-linear constraint evaluations directly through simulation (Mackle et al. 1995; Savic et al. 1997; López-Ibáñez et al. 2008; Al-Ani and Habibi 2012; Paola et al. 2025). Alternative scheduling paradigms include continuous-time formulations, such as the Smart Dynamic Local Search of Brás et al. (2026), which optimizes the continuous start times and duty-cycle durations of pump operations. Although these heuristic and surrogate-assisted methods (Broad et al. 2010; Rao and Alvarruiz 2007; Ashraf et al. 2024) efficiently discover high-quality schedules without exhaustive enumeration, they cannot provide a mathematical certificate of global optimality.

Exact methods make a different trade-off by retaining a certificate for the discrete problem. Costa et al. (2016) coupled hydraulic simulation with an enumerative Branch-and-Bound (B&B) procedure that branches on the number of active pumps and rejects hydraulically infeasible prefixes. Bonvin et al. (2021) instead developed an LP/NLP B&B framework whose tailored MILP relaxation retains demand, storage, and hydraulic information to provide more informative lower bounds. The former is a specialized simulation-driven enumeration, whereas the latter supports a broader class of network components through a more elaborate mathematical relaxation. Neither approach removes the combinatorial growth of the scheduling problem, and both incur substantial work when the horizon or the surviving search tree grows.

Because exact methods must still explore potentially large search trees, parallel B&B architectures offer a principled approach to reducing wall-clock execution time by exploring search subtrees concurrently across distributed computing ranks (Gendron and Crainic 1994; Li and Wah 1986; Hrga and Povh 2021). For simulation-driven optimization, however, tree parallelization addresses only part of the computational bottleneck; accelerating the search also requires eliminating redundant hydraulic simulations during backtracking.

We address a focused research question: can exact enumeration of hourly schedules for interchangeable parallel pumps be accelerated without weakening individual binary actuation con-

straints? To answer this, we use `epanet-bb`, our open-source fork of the Open Water Analytics EPANET repository, which integrates the distributed-memory B&B solver with a customized EPANET hydraulic engine (Open Water Analytics 2026).

The principal contributions of this work are:

1. an exact symmetry reduction that searches aggregate pump counts while a periodic dynamic-programming layer enforces individual pump actuation limits;
2. a distributed-memory parallel search that explores statically partitioned subtrees while asynchronously sharing incumbent upper bounds; and
3. a hierarchical snapshot mechanism that enables state restoration in time independent of search prefix length.

The proposed framework is evaluated through systematic parallel scaling, component ablation, and numerical accuracy analyses. The remainder of this paper is structured as follows. Section 3 defines the pump scheduling optimization model. Section 4 presents the proposed parallel B&B framework. Section 5 reports the numerical experiments and comparative analysis. Finally, Section 6 provides concluding remarks.

PROBLEM FORMULATION

Consider a WDN over a horizon divided into equal scheduling periods $\mathcal{H} = \{1, \ldots, T\}$. Let $\mathcal{P}$ be the set of N_p fixed-speed pumps, $\mathcal{J}$ the set of demand junctions, $\mathcal{K}$ the set of storage tanks, and $\mathcal{L}$ the set of network pipes. The underlying optimization problem is stated in terms of the binary status of each pump. To enable exact symmetry reduction, we assume that the pumps in $\mathcal{P}$ operate in parallel as an interchangeable group sharing identical hydraulic connections, performance curves, efficiency profiles, and energy tariffs, with no identity-specific start-up costs or dynamic states.

Decision Space and Objective

For pump $j \in \mathcal{P}$ and period $h \in \mathcal{H}$, let

$$x_{j,h} = \begin{cases} 1, & \text{if pump } j \text{ is on during period } h, \\ 0, & \text{otherwise.} \end{cases}$$

The objective is to minimize the total electricity cost over the horizon:

$$\min_{\mathbf{x} \in \{0,1\}^{N_P \times T}} \quad z(\mathbf{x}) = \sum_{h=1}^{T} \sum_{j \in \mathcal{P}} C_{j,h} E_{j,h}(\mathbf{x}_{1:h}), \tag{1}$$

where $C_{j,h} \geq 0$ is the electricity tariff and $E_{j,h}(\mathbf{x}_{1:h}) \geq 0$ is the energy consumed during period h. The dependence on the prefix $\mathbf{x}_{1:h}$ reflects the effect of earlier decisions on tank storage and on the hydraulic state at the beginning of the period. Energy consumption and cost are returned by the extended-period hydraulic simulation rather than approximated algebraically within the search.

Operational Constraints

Let $\mathcal{J}_P \subseteq \mathcal{J}$ denote the junctions with prescribed service-pressure requirements. At every scheduling boundary,

$$P_{i,h}(\mathbf{x}_{1:h}) \geq P_i^{\min} \quad \forall i \in \mathcal{J}_P, \ \forall h \in \mathcal{H}. \tag{2}$$

Tank levels must remain within their operating bounds,

$$L_k^{\min} \leq L_{k,h}(\mathbf{x}_{1:h}) \leq L_k^{\max} \quad \forall k \in \mathcal{K}, \ \forall h \in \mathcal{H}, \tag{3}$$

and the terminal storage must recover its initial level:

$$L_{k,T}(\mathbf{x}) \geq L_{k,0} \quad \forall k \in \mathcal{K}. \tag{4}$$

To limit mechanical wear, each pump may complete at most $NA_{\max}$ on–off cycles over the

periodic horizon:

$$\frac{1}{2}\sum_{h=1}^{T}|x_{j,h}-x_{j,h-1}| \le NA_{\max}, \qquad x_{j,0}=x_{j,T}, \qquad \forall j \in \mathcal{P}. \tag{5}$$

The sum counts both off-to-on and on-to-off transitions, including the transition between periods T and 1. Because the boundary is periodic, every complete cycle contributes two transitions, whereas a pump that remains in the same state across the boundary incurs none.

Hydraulic Feasibility

For a fixed schedule, the hydraulic state is defined by flow conservation, energy conservation, pump characteristics, and tank storage dynamics (Bonvin et al. 2021). Let $q_{a,h} \in \mathbb{R}$ denote the flow through pipe or pump arc $a \in \mathcal{L} \cup \mathcal{P}$ during period h, and let $H_{i,h}$ denote the hydraulic head at node i at the end of that period. For each junction $i \in \mathcal{J}$,

$$\sum_{a\in\delta^-(i)} q_{a,h} - \sum_{a\in\delta^+(i)} q_{a,h} = D_{i,h}, \quad \forall i \in \mathcal{J},\ \forall h \in \mathcal{H}, \tag{6}$$

where $\delta^-(i)$ and $\delta^+(i)$ are the arcs entering and leaving node i, and $D_{i,h} \ge 0$ is its demand rate.

For each pipe $\ell = (u, v) \in \mathcal{L}$, the head difference follows the selected nonlinear loss relation (Brown 2002; Burgschweiger et al. 2009a):

$$H_{u,h} - H_{v,h} = \Phi_\ell(q_{\ell,h}), \quad \forall \ell \in \mathcal{L},\ \forall h \in \mathcal{H}, \tag{7}$$

where $\Phi_\ell(0) = 0$ and the sign of $\Phi_\ell(q)$ follows the flow direction.

For each pump $p = (u, v) \in \mathcal{P}$, its binary status defines the following disjunction (Burgschweiger et al. 2009b; Morsi et al. 2012):

$$\begin{aligned} &x_{p,h} = 0 \implies q_{p,h} = 0, \\ &x_{p,h} = 1 \implies H_{v,h} - H_{u,h} = \Psi_p(q_{p,h}), \end{aligned} \qquad \forall p \in \mathcal{P},\ \forall h \in \mathcal{H}, \tag{8}$$

where $\Psi_p(\cdot)$ is the fixed-speed pump curve. In particular, an inactive pump imposes zero flow

rather than zero head difference across its end nodes.

For the constant-area tanks considered here, net inflow changes the stored level according to

$$\sum_{a\in\delta^-(k)} q_{a,h} - \sum_{a\in\delta^+(k)} q_{a,h} = \frac{A_k}{\Delta t}(L_{k,h} - L_{k,h-1}), \forall k \in \mathcal{K},\ \forall h \in \mathcal{H}, \tag{9}$$

where A_k is the cross-sectional area and Δt is the scheduling-period duration.

Rather than explicitly encoding the nonlinear constraints (6)–(9) in the optimization, our approach evaluates feasibility through hydraulic simulation. For each candidate schedule $\mathbf{x}$ visited during Branch-and-Bound traversal, the simulator computes the hydraulic state (flows, heads, tank levels) by solving the full system (6)–(9) using the Global Gradient Algorithm (Todini and Pilati 1988). We then enforce constraints (2), (3), and (4) based on these simulated values. The actuation constraint (5) is enforced directly during schedule construction.

ENHANCED BRANCH-AND-BOUND

We present a distributed-memory parallel Branch-and-Bound (B&B) framework implemented in `epanet-bb` (Rocha et al. 2026). The framework integrates the combinatorial search and the customized EPANET hydraulic engine within the same implementation. The parallel architecture builds a unified search tree across MPI ranks, which concurrently explore statically allocated top-level subtrees and asynchronously exchange global incumbent upper bounds to accelerate pruning, without migrating search nodes during execution (Gendron and Crainic 1994). Within this architecture, the algorithm combines aggregate pump-count branching, exact periodic disaggregation, cost-based pruning, and prefix-independent hydraulic snapshots.

Two-Level Schedule Representation

To reduce the exponential $2^{N_p\times T}$ search space, we define the aggregate decision at each time step $h \in \{1,\dots,T\}$ as

$$y_h = \sum_{j=1}^{N_p} x_{j,h}, \qquad y_h \in \{0,1,\dots,N_p\},$$

where the B&B tree branches on the total number y_h of active pumps rather than on individual binary pump identities. This space reduction is exact for groups of interchangeable parallel pumps that share identical hydraulic connections, performance curves, efficiency profiles, and energy tariffs, with no identity-specific start-up costs or dynamic states. Under these conditions, permuting pump identities leaves both hydraulic feasibility and operational cost invariant.

Because the embedded hydraulic engine requires individual binary statuses to construct and solve the network equations, each aggregate decision y_h must be mapped to a binary state for simulation. We construct a canonical binary representative $\mathbf{x}_h^{\text{can}} = (x_{1,h}^{\text{can}}, \ldots, x_{N_p,h}^{\text{can}})$, which simply activates the first y_h pumps:

$$x_{j,h}^{\text{can}} = \begin{cases} 1, & j \le y_h, \\ 0, & j > y_h. \end{cases}$$

Due to pump interchangeability, simulating $\mathbf{x}_h^{\text{can}}$ yields the exact hydraulic state transition for any binary allocation of y_h active pumps. As illustrated in Fig. 1 for a six-hour sequence, these canonical binary identities serve solely as a standardized input to the hydraulic engine and are not required to satisfy individual actuation limits.

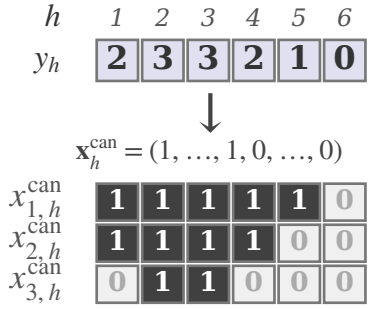

Fig. 1. Two-level schedule representation. The aggregate variable specifies 2, 3, 3, 2, 1, and 0 active pumps over six hours, while $\mathbf{x}_h^{\text{can}}$ supplies the fixed canonical representative simulated hydraulically. Exact disaggregation tracks feasible identity assignments separately and reconstructs the binary actuation schedule.

To enforce individual actuation limits, feasibility is maintained via a dynamic programming layer $\mathcal{D}_h$. For an aggregate prefix $\mathbf{y}_{1:h}$, layer $\mathcal{D}_h$ stores one canonical representative for every reachable permutation class of binary prefixes satisfying $y_t = \sum_{j=1}^{N_p} x_{j,t}$ for $t = 1, \ldots, h$. For each

pump j, the state tuple $(x_{j,1}, x_{j,h}, s_{j,h})$ tracks its initial status at hour 1, its current status at hour h, and the cumulative number $s_{j,h}$ of state switches ($0 \leftrightarrow 1$) accumulated up to hour h. To construct $\mathcal{D}_{h+1}$, the layer transition enumerates binary assignments with exactly y_{h+1} active pumps, updates switch counts, rejects assignments exceeding the $2NA_{\max}$ limit, and canonicalizes the resulting states under pump identity permutations. An aggregate prefix is pruned for actuation infeasibility if and only if its layer $\mathcal{D}_h$ becomes empty. At the terminal step $h = T$, the transition from the final status back to the initial status ($T \rightarrow 1$) is evaluated before accepting a state, thereby enforcing the periodic boundary condition in (5).

To enable full schedule reconstruction, each state retained in layer $\mathcal{D}_h$ stores a pointer to its predecessor state in $\mathcal{D}_{h-1}$ and the permutation applied during canonicalization. Once the optimal aggregate schedule $\mathbf{y}^*$ is identified at the terminal step $h = T$, traversing these predecessor links backward reconstructs an explicit binary schedule $\mathbf{x}^* \in \{0,1\}^{N_p \times T}$ that realizes the aggregate decisions while satisfying every individual pump actuation limit. Consequently, each search node carries two paired components: the hydraulic snapshot generated by the canonical prefix $\mathbf{x}^{\text{can}}_{1:h}$ and the reachable actuation classes $\mathcal{D}_h$.

Proposition (Exact periodic disaggregation). For any aggregate schedule $\mathbf{y}$, the final dynamic-programming layer $\mathcal{D}_T$ (evaluated after the periodic closure transition $T \rightarrow 1$) is nonempty if and only if $\mathbf{y}$ admits a binary disaggregation satisfying the individual periodic actuation limits in (5). Moreover, the predecessor links reconstruct one such binary schedule.

Proof. The result follows by induction on the prefix length. At $h = 1$, the layer $\mathcal{D}_1$ represents every permutation class of binary assignments containing exactly y_1 active pumps. Suppose that $\mathcal{D}_h$ represents all binary prefixes compatible with $\mathbf{y}_{1:h}$ and the accumulated switch limits. The transition to $\mathcal{D}_{h+1}$ considers every assignment containing y_{h+1} active pumps, updates the corresponding switch counts, and rejects only states that already exceed the prescribed limit. Canonicalization merges states that differ solely by a permutation of pump identities. Because their initial status, current status, and switch count coincide up to permutation, they have the same possible successor classes. Thus, $\mathcal{D}_{h+1}$ represents all and only the feasible extensions. At $h = T$, adding the transition from the

final to the initial status ($T \to 1$) enforces the periodic limit. Consequently, the final layer $\mathcal{D}_T$ is nonempty exactly when a feasible binary disaggregation exists, and following its predecessor links reconstructs one. □

By searching over aggregate pump decisions, the framework leverages the permutation invariance of interchangeable pumps. Together with exact periodic disaggregation, this structure guarantees that, for any conclusive execution, the global optimum of the original binary formulation under the specified hydraulic engine configuration is preserved.

Task Decomposition and Parallel Execution

We decompose the search tree into independent subproblems to exploit distributed-memory parallelism. Each *task* is defined by a fixed prefix tuple of the aggregate schedule, $\mathbf{y}_{1:d} = (y_1, \ldots, y_d)$, where d denotes the decomposition depth (typically $d = 8$). This yields $(N_p + 1)^d$ tasks. Fig. 2 illustrates the decomposition: prefixes are enumerated at depth d, and each resulting subtree is assigned to an MPI rank for independent exploration.

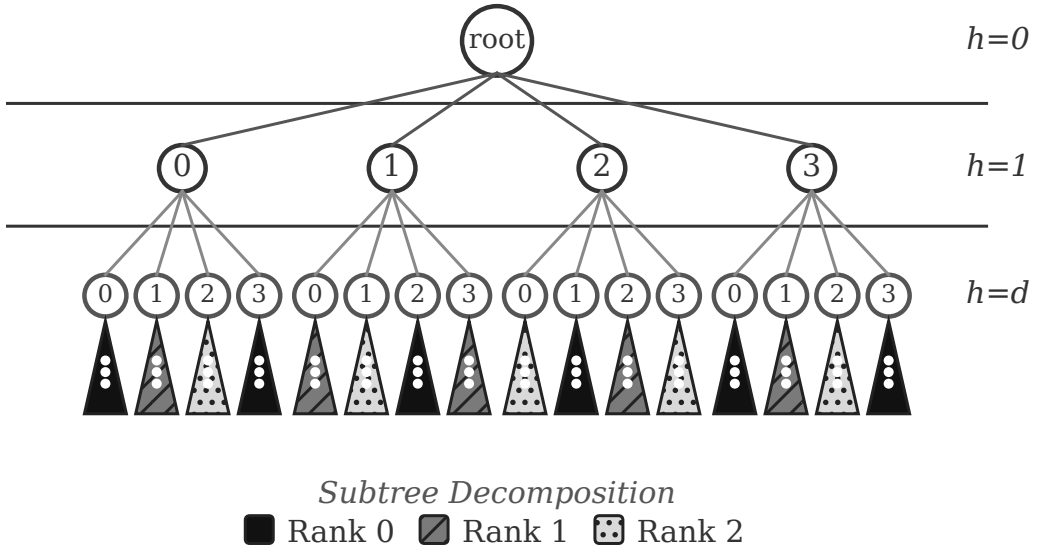

Fig. 2. Task decomposition by prefix. The search tree is partitioned at depth d, yielding $(N_p + 1)^d$ independent subtrees distributed across MPI ranks. In this example, $d = 2$ and $N_p = 3$ generate 16 tasks assigned to 3 ranks via round-robin.

Before assignment, the enumerated tasks are placed in a deterministic fixed-seed permutation

and then distributed round-robin:

$$\text{Rank}(\text{task}_i) = i \bmod N_{\text{procs}}. \tag{10}$$

This task shuffling disperses structurally adjacent prefixes across ranks, mitigating the load imbalance inherent to irregular branch-and-bound trees. However, because static allocation cannot adapt to unequal subtree workloads during search, load imbalance may persist on hard instances.

To enable cooperative pruning without dynamic task migration, MPI ranks explore their assigned subtrees while periodically synchronizing incumbent bounds via non-blocking collective operations. All ranks traverse a shared permuted task sequence, initiating collective bound exchanges at fixed task-index intervals I_{sync}. Between synchronizations, search nodes are pruned whenever their lower bound meets or exceeds $\min\{UB_{\text{local}}, UB_{\text{global}}\}$, where UB_{global} is the most recently received global minimum. This synchronized task-indexing scheme preserves collective call alignment across ranks while allowing independent subtree exploration.

Hydraulic Snapshot Persistence

A primary bottleneck in simulation-driven optimization is the computational cost of evaluating hydraulic constraints. Standard Extended Period Simulation (EPS) computes network states sequentially from $t = 0$ to $t = h$. Consequently, during depth-first tree traversal, backtracking from hour h to $h - 1$ traditionally requires re-simulating the entire hydraulic prefix from $t = 0$—an expensive operation repeated across millions of tree nodes.

Because standard EPANET hydraulic engines do not natively support mid-simulation state restoration, we equipped the embedded engine with a snapshot mechanism that captures the state required to resume simulation at any hourly boundary. A *snapshot* S_h stores the following components:

Node states: hydraulic heads, nodal demands, and junction outflows;

Tank states: water levels, volumes, and surface areas;

Link states: flow rates, head losses, and valve or pump operating statuses;

Energy accumulators: cumulative energy consumption (kWh) and operating costs;

Solver state: simulation timestamp, time-pattern indices, and convergence variables.

Network topology, performance curves, and control rules remain in the rank-local hydraulic model and are not duplicated in each snapshot. Copying a snapshot scales with the stored hydraulic state, but its cost is independent of prefix length h.

During depth-first traversal, a snapshot is retained after each feasible scheduling period. Backtracking restores S_{h-1} before simulating an alternative decision for period h, so restoration is $O(1)$ with respect to prefix length rather than requiring simulation again from $t = 0$. Algorithm 1 makes this dependence explicit by passing S_{h-1} to SIMULATE when advancing one period.

Empirically, disabling snapshot persistence incurs a 9.60× slowdown (Table 1) because hydraulic states must otherwise be reconstructed through repeated simulation from $t = 0$. Snapshots also preserve internal iterative variables alongside physical tank and link states. Under the same numerical configuration, a resumed nonlinear solve therefore starts from the saved hydraulic and solver state.

Algorithm and Pruning Strategies

A fundamental property enabling effective branch-and-bound pruning is the monotonicity of the objective function. Because hourly energy consumption $E_{j,t} \geq 0$ and electricity tariffs $C_{j,t} \geq 0$ are non-negative, the accumulated operating cost up to hour h,

$$z_h = \sum_{t=1}^{h} \sum_{j=1}^{N_p} C_{j,t} \cdot E_{j,t},$$

serves as a monotonic lower bound $LB(\mathbf{y}_{1:h}) = z_h$ along any search trajectory by effectively assigning zero cost to unsimulated future hours $h{+}1 \ldots T$: $z_1 \leq z_2 \leq \cdots \leq z_T$. This non-decreasing property guarantees that if a partial schedule's cost satisfies $z_h \geq \min\{UB_{\text{local}}, UB_{\text{global}}\}$, no further

extension of that prefix can yield a cheaper complete schedule. The corresponding subtree can therefore be safely pruned without compromising global optimality.

Our solver employs Depth-First Search (DFS) and applies the following pruning criteria at each node (hour h):

1. **Actuation Viability:** Prune if the exact disaggregation layer is empty ($\mathcal{D}_h = \emptyset$), including periodic boundary closure at $h = T$.
2. **Tank Saturation:** Prune if engine instrumentation reports corrective blocking of inflow or outflow at tank bounds, rejecting trajectories that depend on artificial saturation.
3. **Tank Bounds:** Prune if $L_{k,h} \notin [L_k^{\min}, L_k^{\max}]$ for any tank k.
4. **Cost Bound:** Prune if $z_h \geq \min\{UB_{\text{local}}, UB_{\text{global}}\}$, as monotonicity ensures no feasible extension can yield a lower total cost.
5. **Pressure Limits:** Prune if $P_{i,h} < P_i^{\min}$ for any junction $i \in \mathcal{J}_P$.
6. **Tank Recovery** (at $h = T$ only): Prune if $L_{k,T} < L_{k,0}$ for any tank k. Intermediate steps are not pruned on this criterion because tank storage can recover during subsequent hours.

Evaluating these pruning criteria in order ensures that the inexpensive combinatorial check ($\mathcal{D}_h = \emptyset$) is executed prior to hydraulic simulation, while the remaining constraints are applied as soon as their required state variables become available. In Algorithm 1, S_h, $\mathcal{D}_h$, and z_h denote the hydraulic snapshot, reachable actuation classes, and accumulated cost after period h, respectively. Hydraulic evaluation determines whether a candidate step is feasible or pruned based on physical constraints and tank saturation instrumentation.

Algorithm 1 outlines the local search kernel executed by each MPI rank on its assigned subtrees, while task distribution and bound synchronization follow the task-indexing protocol described above. In the pseudocode, Next computes the disaggregation layer transition $\mathcal{D}_h$, Close enforces periodic boundary closure at $h = T$, and Canonical constructs the binary representative $\mathbf{x}_h^{\text{can}}$ evaluated by Simulate.

We adopt Depth-First Search (DFS) rather than the Breadth-First Search (BFS) strategy used

by Costa et al. (2016) for two methodological reasons. First, DFS reaches full 24-hour horizons rapidly, establishing an incumbent upper bound early in the search to accelerate cost-based pruning. Second, DFS requires active search frontier memory proportional to the tree depth $O(T)$ rather than the exponential tree width $O((N_p + 1)^T)$, which is essential for parallel scalability across distributed-memory nodes.

Algorithm 1 Local Branch-and-Bound Search of an Assigned Subtree

Require: Assigned prefix tuple $\mathbf{y}_{1:d}$
$S_d, \mathcal{D}_d, z_d,$ feasible $\leftarrow$ AdvancePrefix$(\mathbf{y}_{1:d})$ ▷ Evaluate task prefix
if not feasible **then return**
end if
DFS$(d+1, S_d, \mathcal{D}_d, z_d)$
procedure DFS$(h, S_{h-1}, \mathcal{D}_{h-1}, z_{h-1})$
  if $z_{h-1} \geq \min(UB_{\text{local}}, UB_{\text{global}})$ **then**
    return ▷ Cost bound pruning
  end if
  for $y = 0$ to N_p **do**
    $\mathcal{D}_h \leftarrow$ Next$(\mathcal{D}_{h-1}, y)$ ▷ Update actuation DP layer
    if $h = T$ and $\mathcal{D}_h \neq \emptyset$ **then**
      $\mathcal{D}_h \leftarrow$ Close$(\mathcal{D}_h)$
    end if ▷ Enforce periodic closure
    if $\mathcal{D}_h = \emptyset$ **then**
      continue
    end if ▷ Prune if actuation layer empty
    $\mathbf{x}_h^{\text{can}} \leftarrow$ Canonical(y)
    $S_h, z_h,$ feasible $\leftarrow$ Simulate$(S_{h-1}, \mathbf{x}_h^{\text{can}})$ ▷ EPANET hydraulic step
    if not feasible **then**
      continue
    end if ▷ Prune if hydraulics infeasible
    if $h < T$ **then**
      DFS$(h+1, S_h, \mathcal{D}_h, z_h)$
    else if TanksRecovered(S_h) **then**
      $UB_{\text{local}} \leftarrow z_h$; $\mathbf{y}^* \leftarrow \mathbf{y}_{1:T}$; $\mathbf{x}^* \leftarrow$ Witness$(\mathcal{D}_h)$ ▷ Update incumbent solution
    end if
  end for
end procedure

NUMERICAL EXPERIMENTS

We implemented the proposed framework as a C++17 solver using MPI for distributed computation. All performance experiments were conducted on a Linux server equipped with a 64-core AMD EPYC 9554 processor and 384 GB of RAM, utilizing up to 64 MPI ranks (one rank per physical core). Unless noted otherwise, hydraulic simulations used the baseline relative accuracy of 10^{-4} explicitly specified in the network input file.

Case Study Network

We evaluate our framework on the AnyTown Modified network, a widely adopted benchmark originally proposed by Walski et al. (1987) and later modified by Rao and Salomons (2007) to increase optimization complexity. The network topology (Fig. 3) consists of 23 nodes—19 demand junctions, three storage tanks, and one reservoir—connected by 41 pipes and three pump links. The tanks at nodes 65, 165, and 265 have operating level bounds between 66.53 m and 71.53 m. Water is supplied from the reservoir at node 10 by three identical, fixed-speed pumps operating in parallel (Pumps 111, 222, and 333), while demand nodes share a single temporal profile scaled by individual multipliers. Under the model used here, the pumps have identical hydraulic connections, curves, efficiencies, and energy prices, with no identity-specific start-up cost or dynamic state; they therefore form one interchangeable group. Operating costs are computed using the hourly energy tariff pattern shown in Fig. 4. Although small, this benchmark captures parallel pumping, storage constraints, and time-varying electricity tariffs, making it useful for controlled pump-scheduling experiments.

Parameter Tuning

To identify the most efficient execution parameters, we conducted an exhaustive grid search using Optuna 4.6 with a serial `GridSampler` (Akiba et al. 2019). We tuned three key parameters—the number of MPI ranks ($N_{\text{procs}} \in \{8, 16, 32, 64\}$), the tree decomposition depth ($d \in \{8, 9, 10\}$), and the synchronization interval ($I_{\text{sync}} \in \{1024, 2048, 4096, 8192, 16384, 32768, 65536\}$, defined as the number of top-level task indices between global bound exchanges)—on the 24-hour benchmark instance under $NA_{\max} = 2$. The best performance was achieved by $N_{\text{procs}} = 64$, $d = 8$, and $I_{\text{sync}} = 32768$, with median runtimes of 24.502 s.

Ablation Study

To assess the individual contribution of each active algorithmic component, we performed an ablation study on the 24-hour benchmark problem ($NA_{\max} = 3$) using 64 MPI ranks. Starting from the tuned baseline configuration ($d = 8$, $I_{\text{sync}} = 32768$), four variants each disabled exactly one feature. Across three sequential repetitions, all 15 executions—the baseline and four ablated

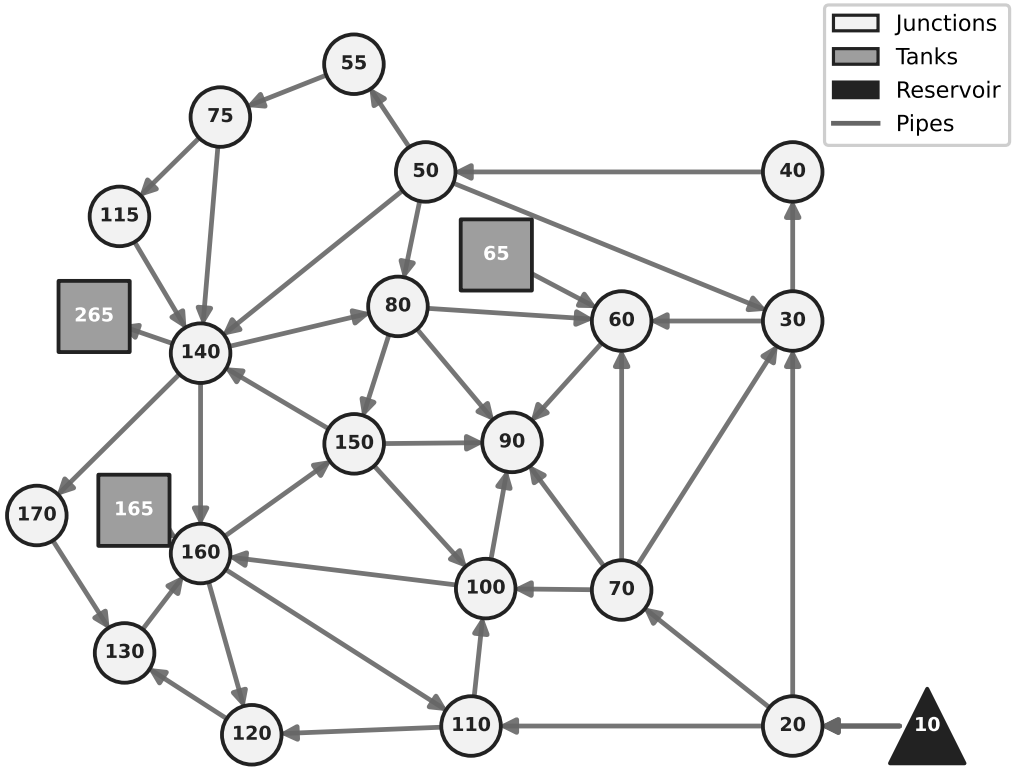

Fig. 3. Topology of the AnyTown Modified network. The network includes a reservoir (node 10), three storage tanks (nodes 65, 165, 265), and 19 demand junctions connected by 41 pipes and three pump links.

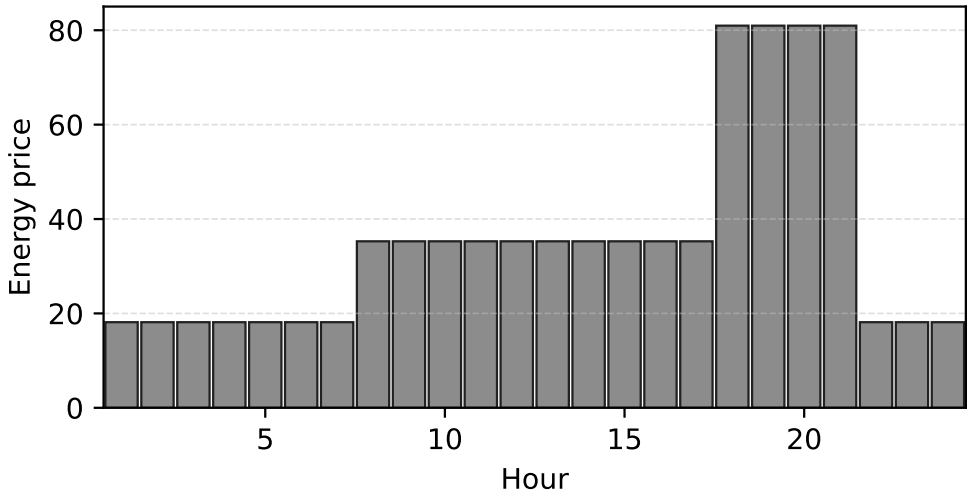

Fig. 4. Hourly energy price pattern used for pump operating cost calculations in the AnyTown Modified network.

variants—completed conclusively and converged to the same global optimum. Table 1 summarizes the results, reporting the median wall-clock time and the relative range, defined as $100(t_{\max} - t_{\min})/t_{\text{median}}$, where $t_{\min}$, t_{median}, and $t_{\max}$ are the minimum, median, and maximum times across the three repetitions. The table also reports the median slowdown relative to the baseline and the median, across repetitions, of the load imbalance among ranks. Load imbalance is expressed as the percentage by which the maximum rank execution time exceeds the mean.

TABLE 1. Ablation study on the 24-hour problem ($NA_{\max} = 3$, $N_{\text{procs}} = 64$).

	Wall time (s)			
Configuration	Median	Range (%)	Slowdown	Load imb. (%)
All features	433	1.2	1.00×	49.3
No cost pruning	11578	4.2	26.86×	77.5
No snapshots	4139	4.7	9.60×	41.4
No task shuffling	2535	0.9	5.84×	701.2
No global synchronization	444	1.4	1.03×	50.8

Cost pruning had the largest measured effect: the no-cost-pruning variant was 26.86× slower than the baseline. Without snapshots, median runtime increased by a factor of 9.60, indicating that hydraulic-state restoration avoids substantial repeated work during backtracking. The no-task-shuffling variant was 5.84× slower than the baseline and increased median load imbalance from 49.3% to 701.2% (Table 1). This result is consistent with the role of task shuffling in distributing static top-level tasks more evenly across ranks. By contrast, the variant without global synchronization was only 2.8% slower. Although small for this network, process count, and tuned configuration, this effect does not preclude benefits in other settings; we therefore retain global synchronization to support timely incumbent sharing across ranks.

Parallel Scalability

TABLE 2. Strong-scaling results for the 24-hour problem ($NA_{\max} = 2$).

	Wall time (s)				
N_{procs}	Median	Range (%)	Speedup	Eff. (%)	Load imb. (%)
1	700.85	0.5	1.00	100.0	0.0
2	355.12	0.8	1.97	98.7	0.9
4	192.91	0.5	3.63	90.9	8.5
8	111.82	0.6	6.27	78.3	24.9
16	62.29	1.9	11.25	70.3	36.2
32	34.04	1.1	20.50	64.1	44.6
64	24.44	0.2	28.68	44.8	87.7

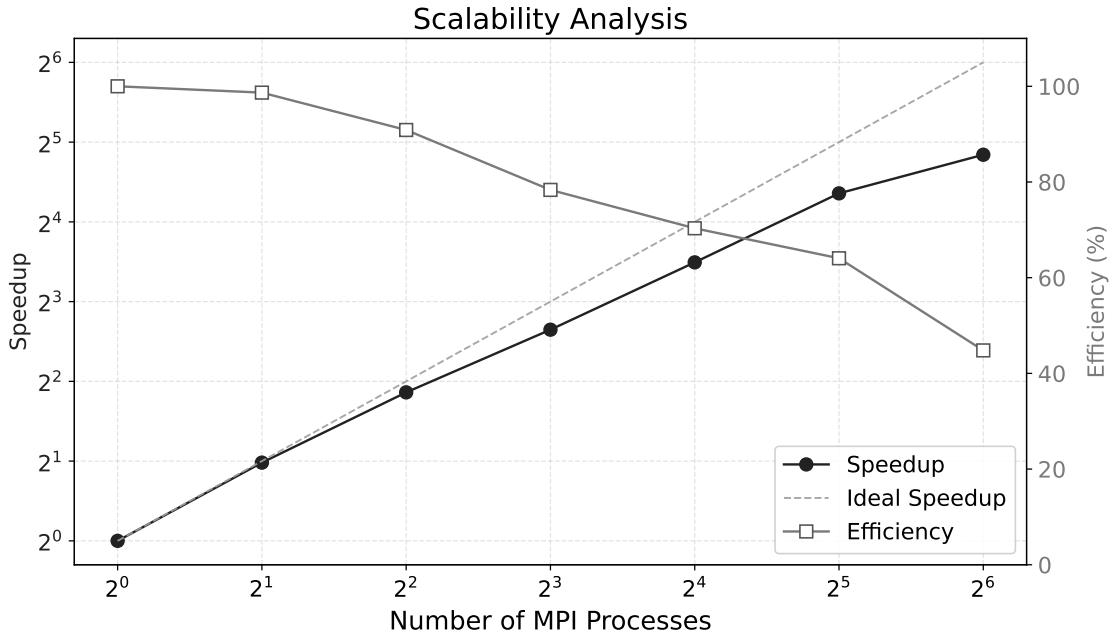

Fig. 5. Strong scaling on AnyTown Modified ($T = 24$, $NA_{\max} = 2$).

As illustrated in Fig. 5 and detailed in Table 2, the solver exhibits monotonic runtime reductions up to 64 physical cores. Median wall-clock time drops from 700.85 s on a single rank to 24.44 s on 64 ranks, achieving a paired median speedup of 28.68× with a parallel efficiency of 44.8%. Thus, speedup remains substantial, while strong-scaling efficiency declines at higher process counts.

This decline is consistent with a limitation of parallel B&B identified by Gendron and Crainic (1994). Static allocation cannot react to the realized sizes of the assigned subtrees; correspondingly,

median load imbalance increases from 0.0% to 87.7% between 1 and 64 ranks.

24-Hour Optimization

We solved the complete 24-hour problem for $NA_{\max} = 1, 2, 3$ with 64 ranks, $d = 8$, and $I_{\text{sync}} = 32768$. Table 3 presents the conclusive search results, and Fig. 6 shows the corresponding tank levels and nodal pressures. The upper panels include the initial levels and the operational bounds $L_{\min} = 66.53$ m and $L_{\max} = 71.53$ m for tanks 65, 165, and 265. The lower panels show pressures at nodes 55, 90, and 170 and their respective minimum thresholds of 42, 51, and 30 m.

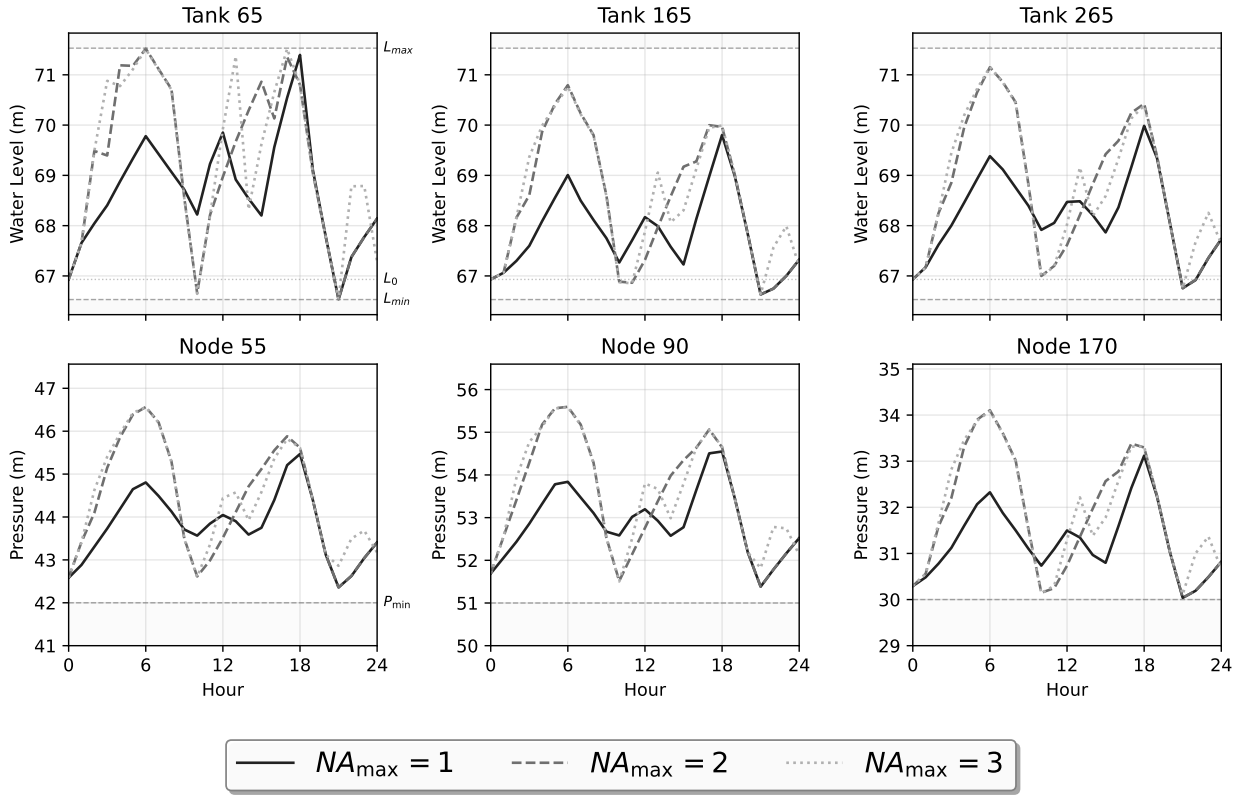

Fig. 6. Hydraulic trajectories for optimal schedules under different actuation limits ($NA_{\max} = 1, 2, 3$). Top: tank water levels; dashed lines show bounds, dotted line shows initial level. Bottom: nodal pressures; dashed lines show minimum thresholds.

For this benchmark, relaxing the actuation limit illustrates a trade-off between operational flexibility and computational effort. Moving from $NA_{\max} = 1$ to $NA_{\max} = 2$ reduces the optimal operating cost from \$3,900.25 to \$3,606.83, but extends wall-clock time from 1.6 to 25.1 s. At $NA_{\max} = 3$, the lowest cost among the three hourly cases is \$3,575.54, obtained in 408.9 s. Load

TABLE 3. Conclusive 24-hour searches on AnyTown Modified with 64 MPI ranks. Load imbalance is the percentage by which the maximum rank execution time exceeds the mean.

$NA_{\max}$	Cost ($)	Time (s)	Load imb. (%)
1	3,900.25	1.6	227.5
2	3,606.83	25.1	92.5
3	3,575.54	408.9	46.0

imbalance also falls from 227.5% to 46.0%, which is consistent with deeper surviving subtrees distributing work more uniformly across the static MPI tasks. Relative to $NA_{\max} = 2$, the final relaxation yields less than 1% in additional savings but requires approximately 16 times more wall-clock time.

Pruning Effectiveness

Table 4 classifies every explored node by its outcome. The total number of nodes grows from 247,695 at $NA_{\max} = 1$ to 350.8 million at $NA_{\max} = 3$, with 66–69% being pruned before branching further into valid partial schedules.

TABLE 4. Node classifications and explored-node counts for the final 24-hour searches.

	$NA_{\max}$		
Classification (% of nodes)	1	2	3
Actuation	52.6	44.3	27.3
Cost bound	4.4	13.0	22.2
Tank levels	7.8	9.3	11.8
Pressure	1.5	2.7	4.7
Total pruned	66.4	69.3	66.0
Not pruned	33.6	30.7	34.0
Explored nodes (count)	247,695	23,165,594	350,839,702

The breakdown of pruning criteria highlights how search dynamics evolve as actuation limits relax. At $NA_{\max} = 1$, actuation constraints dominate the search, accounting for 52.6% of all explored nodes; as $NA_{\max}$ increases to 3, this share declines to 27.3%. Conversely, cost-bound pruning grows from 4.4% to 22.2%, and pressure violations rise from 1.5% to 4.7%, reflecting deeper search depth before sub-optimal or hydraulically infeasible branches are pruned. Tank-level pruning accounts for 7.8%–11.8% of explored nodes. Because the embedded hydraulic engine

may close incident links when continued inflow or outflow would drive a tank beyond its operating bound, standard post-simulation level checks cannot identify every physical overfill or depletion event. We therefore instrumented the engine to capture these internal link-closure events directly, pruning any schedule that relies on automatic simulator intervention to remain within bounds.

Hydraulic Accuracy Sensitivity

To assess numerical sensitivity, we varied the hydraulic convergence tolerance across 10^{-3}, 10^{-4}, and 10^{-7}. The study comprised two complementary experiments. First, each of the three optimal pump schedules was replayed five times at every tolerance while holding all pump decisions fixed, thereby isolating hydraulic solver behavior. Second, the complete problem was reoptimized three times for every combination of $NA_{\max} \in \{1, 2, 3\}$ and tolerance, capturing possible effects on the search tree, selected schedule, and runtime. Both schedule replays and full re-optimizations converged successfully, producing feasible schedules across all tested solver tolerances.

TABLE 5. Hydraulic sensitivity of fixed schedules across convergence tolerances.

$NA_{\max}$	Accuracy	Hyd. trials	Hyd. time (ms)	Time ratio	Max. cost Δ ($)
1	10^{-3}	283	0.849	0.931	3.365×10^{-4}
1	10^{-4}	309	0.925	1.000	0
1	10^{-7}	361	1.034	1.131	1.328×10^{-6}
2	10^{-3}	273	0.821	0.926	6.958×10^{-5}
2	10^{-4}	300	0.884	1.000	0
2	10^{-7}	352	1.010	1.147	1.713×10^{-6}
3	10^{-3}	255	0.771	0.927	5.587×10^{-5}
3	10^{-4}	279	0.830	1.000	0
3	10^{-7}	328	0.950	1.139	1.920×10^{-6}

The results of the sensitivity analysis are summarized in Tables 5 and 6. For fixed schedules (Table 5), relative to the 10^{-4} baseline, median hydraulic execution time decreases by approximately 7% at 10^{-3} and increases by 13–15% at 10^{-7}. The median cumulative number of hydraulic iterations (*trials*) decreases by 8–9% at 10^{-3} and increases by 17–18% at 10^{-7}. Because these sub-millisecond measurements isolate the hydraulic kernel, Table 6 evaluates the end-to-end optimization impact. Tightening the accuracy to 10^{-7} increases total wall-clock time by roughly 8% for $NA_{\max} = 2$ and 3, where solver computation dominates MPI launch overhead. Although tree-traversal node

TABLE 6. End-to-end optimization sensitivity across convergence tolerances.

$NA_{\max}$	Accuracy	Median (s)	Time ratio	Cost ($)	Nodes	Distinct schedules
1	10^{-3}	1.567	0.969	3,900.25	247,666	1
1	10^{-4}	1.617	1.000	3,900.25	247,601	1
1	10^{-7}	1.617	1.000	3,900.25	247,638	1
2	10^{-3}	24.703	0.955	3,606.83	23,169,013	1
2	10^{-4}	25.855	1.000	3,606.83	23,167,485	1
2	10^{-7}	27.958	1.081	3,606.83	23,170,581	1
3	10^{-3}	407.838	0.958	3,575.54	350,650,672	1
3	10^{-4}	427.881	1.000	3,575.54	350,688,093	1
3	10^{-7}	462.232	1.080	3,575.54	350,673,687	1

counts exhibit minor floating-point variations, all nine optimizations for each $NA_{\max}$ select the same optimal schedule. Accuracy-dependent costs differ by at most 3.36×10^{-4} and therefore agree at the reported cent precision. We retain 10^{-4} as the reference used in the main experiments.

Comparative Analysis

To contextualize solver performance within the literature, we evaluate published approaches at two distinct levels: a direct replay of binary schedules under a unified physical protocol, and a methodological assessment of approaches based on alternative optimization formulations.

For the binary schedule evaluation, we replayed the 24-hour pump decisions reported by Costa et al. (2016) (sequential B&B), Cimorelli et al. (2020) (Genetic Algorithm), and Paola et al. (2025) (Digital Harmony Search). Each schedule was re-simulated using the embedded hydraulic engine with the same 24-hour periodic boundary check and physical tank-saturation policy, allowing physical feasibility and operating costs to be assessed under identical hydraulic conditions.

The evaluation outcomes are summarized in Table 7 and illustrated in Figs. 7–9. All three Costa et al. schedules and all three schedules produced by the proposed solver satisfy the physical and operational constraints. In contrast, the schedules of Cimorelli et al. (2020) exceed the periodic actuation limit during the closing transition from hour 24 to hour 1. The schedule of Paola et al. (2025) is feasible for $NA_{\max} = 1$, but the schedules for $NA_{\max} = 2$ and 3 require tank-saturation interventions at hour 21 and are therefore infeasible under the common physical policy. Within the feasible subset, the proposed solver produced operating costs 0.287%, 0.325%, and 0.088% lower

TABLE 7. Evaluation of nine published pump schedules under the common physical feasibility policy.

Source	$NA_{\max}$	Feasible	Reason	Published ($)	Replay ($)
Costa et al.	1	Yes	–	3,916.98	3,916.98
Costa et al.	2	Yes	–	3,618.59	3,618.60
Costa et al.	3	Yes	–	3,578.67	3,578.67
Cimorelli et al.	1	No	Periodic actuation	3,634.67	–
Cimorelli et al.	2	No	Periodic actuation	3,580.11	–
Cimorelli et al.	3	No	Periodic actuation	3,575.54	–
De Paola et al.	1	Yes	–	3,911.52	3,911.48
De Paola et al.	2	No	Tank saturation	3,606.22	–
De Paola et al.	3	No	Tank saturation	3,577.40	–

than the lowest feasible external schedule for $NA_{\max} = 1, 2, 3$, respectively. Infeasible published schedules are retained in the figures for identification, labeled with their reported costs, but are excluded from the cost comparison among feasible schedules. This schedule audit does not rank the original optimization algorithms, which were not rerun under a common computational protocol.

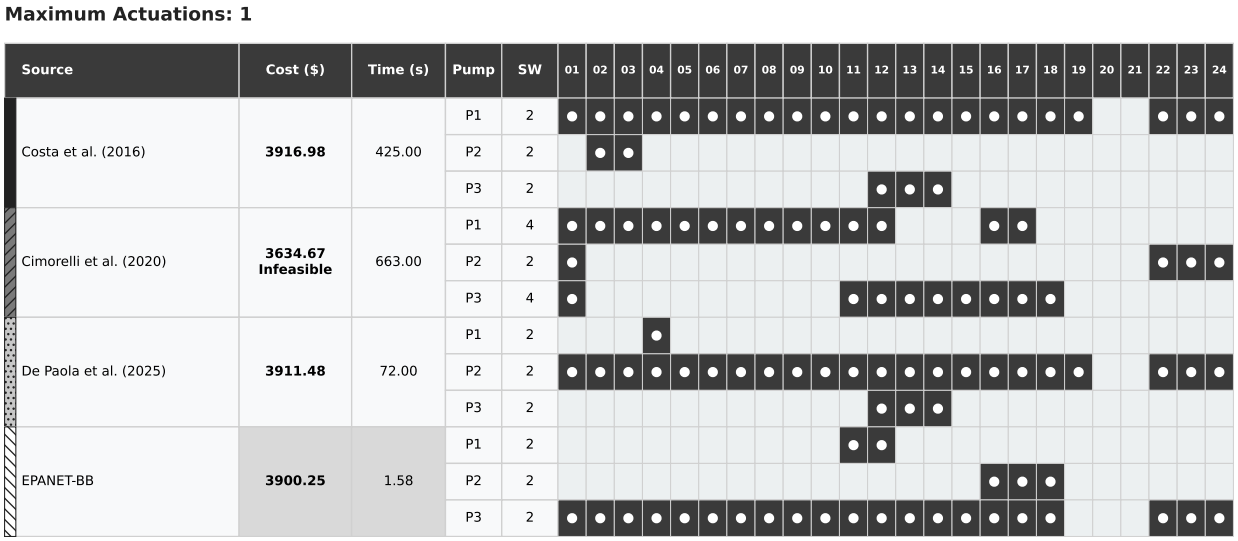

Fig. 7. Published schedules and exact replay status for $NA_{\max} = 1$. Costs are comparable only among schedules marked feasible.

Beyond uniform binary schedule replays, alternative optimization formulations provide a broader methodological context. Bonvin et al. (2021) developed an exact LP/NLP branch-and-bound framework that uses a tailored MILP relaxation based on convex linear outer approximations

Maximum Actuations: 2

Source	Cost ($)	Time (s)	Pump	SW	01	02	03	04	05	06	07	08	09	10	11	12	13	14	15	16	17	18	19	20	21	22	23	24
Costa et al. (2016)	**3618.60**	36914.00	P1	4	●	●	●	●	●	●	●	●	●		●	●	●	●	●	●	●	●				●	●	●
			P2	4		●	●								●	●	●											
			P3	4															●		●							
Cimorelli et al. (2020)	**3580.11 Infeasible**	663.00	P1	6	●		●	●	●	●	●	●								●	●							
			P2	4											●	●	●	●								●	●	
			P3	4	●	●	●								●	●	●	●	●	●	●	●						
De Paola et al. (2025)	**3606.22 Infeasible**	904.00	P1	4	●	●	●	●	●	●	●	●			●	●	●	●	●	●	●	●				●	●	●
			P2	4		●	●								●	●	●	●	●									
			P3	2																	●							
EPANET-BB	**3606.83**	25.06	P1	4		●		●																				
			P2	4											●	●	●	●	●		●							
			P3	4	●	●	●	●	●	●	●	●			●	●	●	●	●	●	●	●				●	●	●

Fig. 8. Published schedules and exact replay status for $NA_{\max} = 2$. Infeasible schedules retain their published costs for identification, not comparison.

Maximum Actuations: 3

Source	Cost ($)	Time (s)	Pump	SW	01	02	03	04	05	06	07	08	09	10	11	12	13	14	15	16	17	18	19	20	21	22	23	24
Costa et al. (2016)	**3578.67**	292032.00	P1	6	●	●	●	●	●	●	●	●			●	●	●	●	●	●	●	●				●	●	
			P2	6		●		●							●	●	●	●	●									
			P3	4																	●					●		
Cimorelli et al. (2020)	**3575.54 Infeasible**	663.00	P1	8	●	●	●	●	●	●	●	●			●	●	●		●	●	●					●	●	
			P2	8	●			●							●	●	●		●	●	●							
			P3	6												●	●					●				●		
De Paola et al. (2025)	**3577.40 Infeasible**	960.00	P1	6	●	●	●	●	●	●	●	●			●	●	●	●	●	●	●	●				●	●	
			P2	6	●		●								●	●	●	●	●									
			P3	4																	●					●		
EPANET-BB	**3575.54**	408.95	P1	6		●	●								●	●	●									●		
			P2	6												●	●		●	●	●					●	●	
			P3	6	●	●	●	●	●	●	●	●			●	●	●		●	●	●	●						

Fig. 9. Published schedules and exact replay status for $NA_{\max} = 3$. Infeasible schedules retain their published costs for identification, not comparison.

to obtain lower bounds and evaluates integer candidates through restricted nonlinear subproblems. For binary-setting networks such as AnyTown Modified, this candidate evaluation is specialized to an extended-period hydraulic feasibility check. In their historical comparison with the sequential enumeration of Costa et al. (2016), the LP/NLP method was slightly slower in the most restricted case and retained optimality gaps of 1.6% and 1.9% in the two less restricted cases after the corresponding comparison times. These results show that more informative lower bounds do not ensure faster solution on every instance. They do not constitute a direct comparison with the present implementation because the hardware, hydraulic models, formulations, and switching conventions differ.

A complementary methodological direction replaces the hourly binary representation with continuous pump-operation intervals. Brás et al. (2026) considered a time-position-unrestricted duty-cycle formulation in which each pump operation is described by a real-valued starting time and duration and optimized it using the hybrid Smart Dynamic Local Search (Smart-DLS) heuristic. For AnyTown Modified, the authors reported a best operating cost of \$3,416.18 and an estimated optimization time of approximately 5.14 minutes. The numerically lower published cost is consistent with greater temporal flexibility, but it cannot be attributed to that flexibility alone or interpreted as algorithmic dominance because the number and distribution of pump starts, the feasible space, and the hydraulic verification protocol differ from those of the hourly cases considered here. Smart-DLS provides no certificate of global optimality.

In summary, the direct schedule replay provides a controlled comparison of discrete hourly schedules under common physical rules. The contextual comparisons with Bonvin et al. (2021) and Brás et al. (2026) instead illustrate how temporal representation, bounding information, and search strategy lead to different balances among flexibility, computational effort, and optimality guarantees.

Discussion

Ablation Scope. Cost pruning, snapshots, and task shuffling have clear effects in the tested configuration, whereas global synchronization has only a small measured effect. Because the

experiment changes one active feature at a time on one network, these results characterize the tested setting and do not establish the same relative contributions elsewhere.

Scalability Considerations. The AnyTown Modified network has three pumps, yielding $4^{24} \approx 2.8 \times 10^{14}$ unconstrained high-level action sequences over 24 hours. Networks with more pumps will face exponentially larger search spaces. In the AnyTown experiments, the actuation constraint $NA_{\max}$ provides the largest individual share of pruning. For networks where operational constraints are looser, additional strategies such as bound tightening, symmetry breaking, or hierarchical decomposition may be necessary.

Parallel Efficiency. For $NA_{\max} = 2$, efficiency declines from 98.7% at two ranks to 44.8% at 64 ranks while wall time continues to decrease. The accompanying growth in load imbalance and candidate work indicates that static decomposition, pruning, and incumbent timing interact. The parallel B&B strategies surveyed by Gendron and Crainic (1994) include dynamic redistribution on request and organizations combining local and global work pools. Such strategies, including work stealing, are natural extensions to evaluate, but they also introduce communication, incumbent-coordination, and termination-detection costs. Their net benefit therefore depends on task granularity and must be measured.

Numerical and Comparative Limits. The accuracy study supports the stability of the selected schedules on this instance over the tested tolerance range, but it is not a general conditioning analysis of the embedded hydraulic engine or of hydraulic simulators more broadly. The external comparison evaluates reported schedules under a common policy and does not rerun the original optimization methods. These distinctions limit the claims to the evidence actually reproduced.

Generalizability. The experiments use one benchmark with three interchangeable fixed-speed pumps. The implementation is not restricted to this particular topology, but search complexity remains exponential and performance on larger networks or groups with more pumps must be established empirically.

CONCLUSION

We developed a distributed-memory B&B framework for the hourly scheduling of interchange-

able parallel fixed-speed pumps. The method searches aggregate pump-count sequences while periodic dynamic programming retains exactly those sequences that admit a binary disaggregation satisfying the individual actuation limits. An embedded hydraulic engine evaluates each branch incrementally, and hydraulic snapshots avoid prefix re-simulation during backtracking. Cost-based pruning and parallel exploration then reduce the work and wall-clock time of the search. Whenever all required hydraulic evaluations converge, the reconstructed schedule is a global optimum of the specified discrete problem under the fixed hydraulic model and numerical configuration.

On the tested benchmark, the experiments show how these mechanisms affect both operating cost and computational effort. All three 24-hour cases completed conclusively with 64 MPI ranks. Relaxing the actuation limit reduced operating cost but increased the surviving search space and wall-clock time, showing that operational flexibility and the effort required to certify a schedule must be considered together. For the intermediate case, strong scaling reduced the median runtime from 700.85 s with one rank to 24.44 s with 64 ranks, although parallel efficiency declined as load imbalance and candidate work increased. The ablation study further identified cost pruning, hydraulic snapshots, and task shuffling as the main contributors to performance in the tested configuration.

Because the search space still grows exponentially, performance on larger or real networks, multiple pump groups, and variable-speed operation remains to be established. Future work should evaluate dynamic task redistribution, stronger bounds or hierarchical decomposition, extensions to multiple interchangeable groups, and additional networks. Within the scope demonstrated here, the framework provides an exact reference for studying actuation constraints, auditing published schedules, and measuring the quality of faster approximate methods.

DATA AVAILABILITY STATEMENT

Some or all data, models, or code generated or used during the study are available in a repository online in accordance with funder data retention policies. Versioned `epanet-bb` releases archive the source code, benchmark network files, experiment configurations, precomputed results, and reconstruction scripts under the project Zenodo record (Rocha et al. 2026). The development

repository is available at https://github.com/michaelsouza/epanet-bb.

REPRODUCIBLE RESULTS

The computational results reported in this study were reproduced by Michael Souza and Jéssica Gomes Melo da Rocha, both affiliated with the Federal University of Ceará. Using the archived `epanet-bb` repository and the benchmark input files, experiment configurations, and analysis scripts referenced in the Data Availability Statement, they regenerated the optimization outputs and reproduced the tables and figures derived from those outputs, including the ablation, scalability, and benchmark comparison results reported in the manuscript.

ACKNOWLEDGMENTS

This research was partially funded by FAPESP (Grant Nos. 2013/07375-0, 2023/08706-1, and 2024/00923-6). Additional funding was provided by CNPq (Grant Nos. 305227/2022-0, 404616/2024-0, and 402609/2025-5). Michael Souza, Albert Einstein Fernandes Muritiba, and Ascânio Dias Araújo acknowledge financial support from CAPES (Coordenação de Aperfeiçoamento de Pessoal de Nível Superior), CNPq (Conselho Nacional de Desenvolvimento Científico e Tecnológico), and FUNCAP (Fundação Cearense de Apoio ao Desenvolvimento Científico e Tecnológico). Jéssica Gomes Melo da Rocha acknowledges a Master's scholarship from FUNCAP. The funding agencies had no role in the study design; data collection, analysis, or interpretation; manuscript preparation; or the decision to submit the article for publication.

NOTATION

The following symbols are used in this paper:

A_k = cross-sectional area of tank k (m^2);
$C_{j,h}$ = electricity tariff for pump j in scheduling period h (\$/kWh);
d = task-decomposition depth;
$D_{i,h}$ = water demand at node i in scheduling period h (m^3/s);
$\mathcal{D}_h$ = reachable actuation-state classes after period h;
$E_{j,h}$ = energy consumed by pump j during scheduling period h (kWh);
$\mathcal{H}$ = set of scheduling periods in the planning horizon;
$H_{i,h}$ = hydraulic head at node i in scheduling period h (m);
I_{sync} = global task-index interval between incumbent exchanges;
$\mathcal{J}$ = set of demand junctions;
$\mathcal{J}_P$ = set of junctions with minimum-pressure requirements;
$\mathcal{K}$ = set of storage tanks;
$LB(\mathbf{y}_{1:h})$ = lower bound for aggregate prefix $\mathbf{y}_{1:h}$;
$L_{k,h}$ = water level in tank k in scheduling period h (m);
$L_k^{\min}, L_k^{\max}$ = minimum and maximum operating levels of tank k (m);
$\mathcal{L}$ = set of pipes;
$NA_{\max}$ = maximum number of periodic on–off cycles per pump;
N_p = number of pumps;
N_{procs} = number of MPI ranks;
$\mathcal{P}$ = set of pumps;
$P_{i,h}$ = pressure at node i in scheduling period h (m);
$P_i^{\min}$ = minimum required pressure at junction i (m);
$q_{a,h}$ = flow rate through arc a in scheduling period h (m^3/s);
$s_{j,h}$ = cumulative state switches of pump j through period h;
S_h = hydraulic snapshot after scheduling period h;
T = number of scheduling periods in the planning horizon;
UB_{global} = global upper bound on total operating cost;
UB_{local} = local upper bound on total operating cost;
$x_{j,h}$ = binary status of pump j in scheduling period h;
$\mathbf{x}_h^{\text{can}}$ = canonical binary representative in period h;
y_h = number of active pumps in scheduling period h;
z_h = accumulated operating cost through period h;
Δt = scheduling-period duration (s);
$\Phi_\ell(\cdot)$ = head-loss relation for pipe ℓ;
$\Psi_p(\cdot)$ = head-gain relation for pump p.

REFERENCES

Akiba, T., Sano, S., Yanase, T., Ohta, T., and Koyama, M. (2019). "Optuna: A next-generation hyperparameter optimization framework." *Proceedings of the 25th ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, KDD '19, ACM, 2623–2631 (July).

Al-Ani, D. and Habibi, S. (2012). "Optimal pump operation for water distribution systems using

a new multi-agent particle swarm optimization technique with EPANET." *2012 25th IEEE Canadian Conference on Electrical and Computer Engineering (CCECE)*, IEEE, 1–6 (April).

Ashraf, I., Strotherm, J., Hermes, L., and Hammer, B. (2024). "Physics-informed graph neural networks for water distribution systems." *Proceedings of the AAAI Conference on Artificial Intelligence*, 38(20), 21905–21913.

Bonvin, G., Demassey, S., and Lodi, A. (2021). "Pump scheduling in drinking water distribution networks with an LP/NLP-based branch and bound." *Optimization and Engineering*, 22(3), 1275–1313.

Brás, M., Moura, A., and Andrade-Campos, A. (2026). "A novel hybrid optimization approach for cost-efficient pump scheduling in water supply systems." *Omega*, 138, 103378.

Broad, D. R., Maier, H. R., and Dandy, G. C. (2010). "Optimal operation of complex water distribution systems using metamodels.." *Journal of Water Resources Planning and Management*, 136(4), 433–443.

Brown, G. O. (2002). "The history of the Darcy-Weisbach equation for pipe flow resistance." *Environmental and Water Resources History*, Reston, VA, American Society of Civil Engineers, 34–43 (October).

Burgschweiger, J., Gnädig, B., and Steinbach, M. C. (2009a). "Nonlinear programming techniques for operative planning in large drinking water networks." *Open Applied Mathematics Journal*, 3(1), 14–28.

Burgschweiger, J., Gnädig, B., and Steinbach, M. C. (2009b). "Optimization models for operative planning in drinking water networks." *Optimization and Engineering*, 10(1), 43–73.

Cimorelli, L., D'Aniello, A., and Cozzolino, L. (2020). "Boosting genetic algorithm performance in pump scheduling problems with a novel decision-variable representation." *Journal of Water Resources Planning and Management*, 146(5), 04020023.

Costa, L. H. M., de Athayde Prata, B., Ramos, H. M., and de Castro, M. A. H. (2016). "A branch-and-bound algorithm for optimal pump scheduling in water distribution networks." *Water Resources Management*, 30(3), 1037–1052.

Fooladivanda, D. and Taylor, J. A. (2018). "Energy-optimal pump scheduling and water flow." *IEEE Transactions on Control of Network Systems*, 5(3), 1016–1026.

Gendron, B. and Crainic, T. G. (1994). "Parallel branch-and-bound algorithms: Survey and synthesis." *Operations Research*, 42(6), 1042–1066.

Hrga, T. and Povh, J. (2021). "MADAM: a parallel exact solver for max-cut based on semidefinite programming and ADMM." *Computational Optimization and Applications*, 80(2), 347–375.

Lansey, K. E. and Awumah, K. (1994). "Optimal pump operations considering pump switches." *Journal of Water Resources Planning and Management*, 120(1), 17–35.

Li, G.-J. and Wah, B. W. (1986). "Coping with anomalies in parallel branch-and-bound algorithms." *IEEE Transactions on Computers*, C-35(6), 568–573.

López-Ibáñez, M., Prasad, T. D., and Paechter, B. (2008). "Ant colony optimization for optimal control of pumps in water distribution networks." *Journal of Water Resources Planning and Management*, 134(4), 337–346.

Mackle, G., Savic, D. A., and Walters, G. A. (1995). "Application of genetic algorithms to pump scheduling for water supply." *First International Conference on Genetic Algorithms in Engineering Systems: Innovations and Applications (GALESIA)*, Vol. 1995, IEE, 400–405.

Morsi, A., Geißler, B., and Martin, A. (2012). "Mixed integer optimization of water supply networks." *Mathematical Optimization of Water Networks*, Springer, 35–54.

Open Water Analytics (2026). "OWA-EPANET: Water distribution system hydraulic and water quality analysis toolkit. Software repository, accessed July 28, 2026.

Paola, F. D., Pugliese, F., Fontana, N., and Giugni, M. (2025). "A new digital harmony search algorithm for optimizing pump scheduling in water distribution networks." *Water Research X*, 27, 100300.

Rao, Z. and Alvarruiz, F. (2007). "Use of an artificial neural network to capture the domain knowledge of a conventional hydraulic simulation model." *Journal of Hydroinformatics*, 9(1), 15–24.

Rao, Z. and Salomons, E. (2007). "Development of a real-time, near-optimal control process for

water-distribution networks." *Journal of Hydroinformatics*, 9(1), 25–37.

Reis, A. L., Lopes, M. A. R., Andrade-Campos, A., and Antunes, C. H. (2023). "A review of operational control strategies in water supply systems for energy and cost efficiency." *Renewable and Sustainable Energy Reviews*, 175, 113140.

Rocha, J. G. M., Muritiba, A. E. F., Araújo, A. D., Lavor, C. C., and Souza, M. (2026). "EPANET-BB: Parallel branch-and-bound pump scheduling optimizer. Versioned software archive; development repository at https://github.com/michaelsouza/epanet-bb.

Savic, D. A., Walters, G. A., and Schwab, M. (1997). "Multiobjective genetic algorithms for pump scheduling in water supply." *Workshop on Evolutionary Computing*, Springer, 227–235.

Todini, E. and Pilati, S. (1988). "A gradient algorithm for the analysis of pipe networks." *Computer Applications in Water Supply: Vol. 1—Systems Analysis and Simulation*, B. Coulbeck and C.-H. Orr, eds., Research Studies Press Ltd., Taunton, UK, 1–20.

Verleye, D. and Aghezzaf, E.-H. (2016). "Generalized benders decomposition to reoptimize water production and distribution operations in a real water supply network." *Journal of Water Resources Planning and Management*, 142(2), 04015059.

Walski, T. M., Brill, E. D., Gessler, J., Goulter, I. C., Jeppson, R. M., Lamey, K., Lee, H. L., Liebman, J. C., Mayo, L., Morgan, D. R., and Ormsbee, L. (1987). "Battle of the network models: epilogue." *Journal of Water Resources Planning and Management*, 113(2), 191–203.